NYC Taxi Data Engineering Platform

From monthly source files through Fabric to a refreshing Power BI report

PROJECT OVERVIEW

This project builds a repeatable Fabric data path for NYC yellow taxi trips: monthly Parquet and zone-reference files move through Lakehouse staging, Dataflow Gen2 enrichment, and the Warehouse and semantic model. The completed Power BI report presents trip, passenger, and revenue KPIs with date, payment-method, and vendor filters. Its data refreshes successfully whenever the pipelines run.

DATASET

- **Trip data:** 6 monthly Parquet files, Jan-Jun 2026
- **Reference data:** taxi_zone_lookup.csv
- **Core fields:** pickup/drop-off time and location, vendor, passengers, distance, payment, total amount
- **Enrichment:** location IDs mapped to borough and zone names

TECHNOLOGY STACK

- **Microsoft Fabric** workspace and OneLake
- **Lakehouse** for source file organization
- **Data Factory pipelines** for copy, variables, scripts, and orchestration
- **Dataflow Gen2 / Power Query** for joins and type preparation
- **Fabric Warehouse SQL** for tables and metadata
- **Semantic model** *nyc* + Power BI report

POWER BI REPORT SNAPSHOT

REFRESHES AFTER PIPELINE RUNS

114.19M	16.2M	541.09M
TOTAL TRIPS	PASSENGERS	TOTAL REVENUE

DATE RANGE | PAYMENT METHOD | VENDOR FILTERS

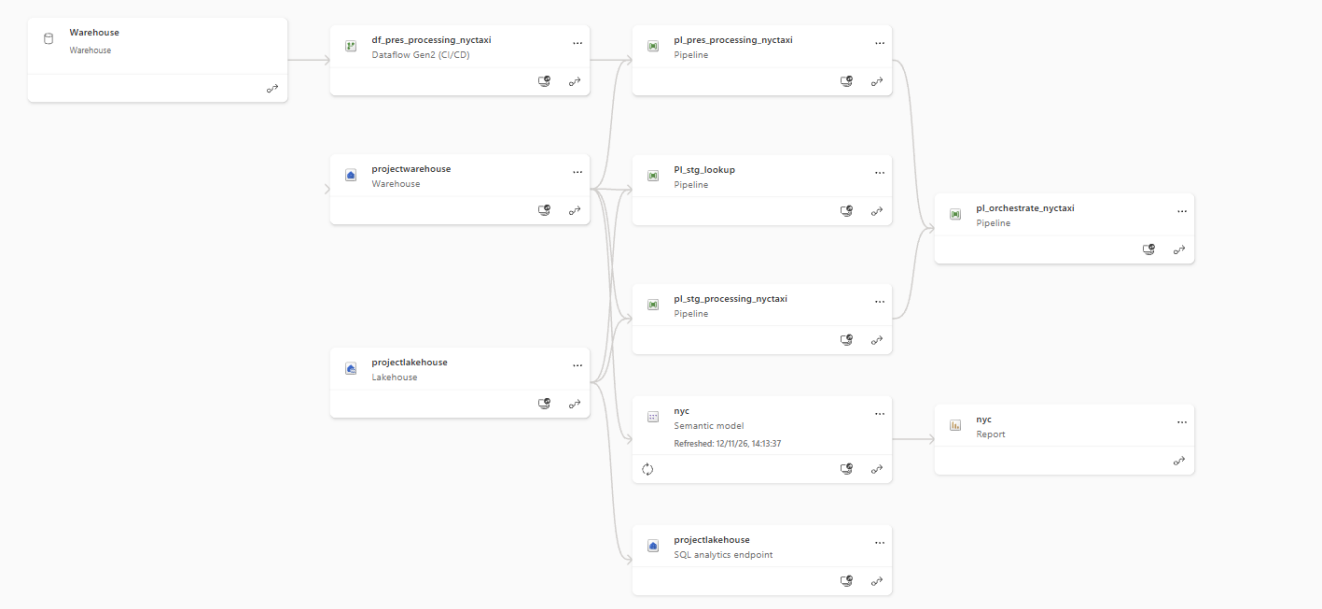
Pipeline status: successful runs refresh the Power BI report data.

Architecture and workflow

Data movement and orchestration shown as separate views

A. FABRIC ITEM LINEAGE

Lineage view connects the Warehouse, Lakehouse, pipelines, semantic model, and Power BI report.



B. EXACT PROCESS SEQUENCE

The orchestration pipeline calls the staging path before the presentation path.

1 INGEST

Place monthly trip Parquet files and taxi zone CSV in Lakehouse Files.

2 LOOKUP STAGE

pl_stg_lookup copies the zone reference to staging.

3 ORCHESTRATE

pl_orchestrate_nyctaxi invokes staging, then presentation.

4 WATERMARK

Script reads latest processed pickup date; variables set v_date / v_end_date.

5 STAGE + CLEAN

Copy trip rows to staging; apply the date window and write staging metadata.

6 ENRICH

Dataflow Gen2 merges trips with zone lookup and prepares column types.

7 PRESENT

Presentation path writes prepared data to the Warehouse and logs the load.

8 REPORT

Power BI report uses semantic model nyc and refreshes with each successful pipeline run.

Methodology and SQL

Nine sequential stages through the finished, refreshing BI report



SEQUENTIAL METHODOLOGY

01 Place source files

Six monthly yellow taxi Parquet files and taxi_zone_lookup.csv are organized in Lakehouse Files.

02 Stage the lookup

pl_stg_lookup copies zone reference data to staging so it can enrich trip records.

03 Orchestrate

pl_orchestrate_nyctaxi runs the staging pipeline before the presentation pipeline.

04 Find the watermark

A Script reads the latest processed pickup date to guide incremental processing.

05 Set the load window

Variables v_date and v_end_date define the range; Copy data loads trip rows to staging.

06 Clean and audit

Date cleanup removes out-of-window rows; the staging metadata procedure records counts and latest date.

07 Enrich the trips

Dataflow Gen2 merges trips with the zone lookup and assigns appropriate column types.

08 Prepare presentation

The presentation pipeline runs the Dataflow and writes a presentation load audit entry.

09 Refresh the BI report

The Power BI report uses model nyc, with date, payment, and vendor filters plus trip, passenger, and revenue KPIs. Pipeline runs refresh its data.

SQL QUERIES AND PURPOSE

QUERY 1 stg.data_cleaning_stg

```
CREATE PROCEDURE stg.data_cleaning_stg
@end_date DATETIME2, @start_date DATETIME2
AS DELETE FROM stg.nyctaxi_yellow
WHERE tpep_pickup_datetime < @start_date
 OR tpep_pickup_datetime > @end_date;
```

Deletes rows before the start or after the end; both boundary dates remain.

QUERY 2 Initial audit check and metadata setup

```
SELECT 'placeholder' AS pipeline_run_id,
 'staging_nyctaxi_yellow' AS table_processed,
 COUNT(*) AS row_processsed,
 MAX(tpep_pickup_datetime) AS latest_processed_pickup,
 CURRENT_TIMESTAMP AS processed_datetime
FROM stg.nyctaxi_yellow;
CREATE SCHEMA metadata;
CREATE TABLE metadata.processing_log (
pipeline_run_id VARCHAR(255), table_processed VARCHAR(255),
rows_processed INT, latest_processed_pickup DATETIME2(6),
processed_datetime DATETIME2(6));
```

The SELECT previews row count, latest pickup, and time; the ID is a static placeholder. Schema/table creation sets up persistent audit storage. The pasted alias row_processsed differs from rows_processed.

QUERY 3 metadata.insert_staging_metadata

```
CREATE PROCEDURE metadata.insert_staging_metadata
@pipeline_run_id, @table_name, @processed_date
INSERT INTO metadata.processing_log (...)
SELECT @pipeline_run_id, @table_name, COUNT(*),
MAX(tpep_pickup_datetime), @processed_date
FROM stg.nyctaxi_yellow;
```

Writes one staging audit row with pipeline values plus the count and latest pickup date from staging.

QUERY 4 dbo.nyctaxi_yellow table

```
CREATE TABLE dbo.nyctaxi_yellow (
vendor VARCHAR(50), pickup/dropoff DATE,
pu_borough/pu_zone VARCHAR(100),
do_borough/do_zone VARCHAR(100),
payment_method VARCHAR(50), passenger_count INT,
trip_distance FLOAT, total_amount FLOAT);
```

Defines the presentation schema and types for trip metrics and readable pickup/drop-off geography.

QUERY 5 metadata.insert_presentation_metadata

```
CREATE PROCEDURE metadata.insert_presentation_metadata
@pipeline_run_id, @table_name, @processed_date
INSERT INTO metadata.processing_log (...)
SELECT @pipeline_run_id, @table_name, COUNT(*),
MAX(tpep_pickup_datetime), @processed_date
FROM dbo.nyctaxi_yellow;
SELECT * FROM metadata.processing_log;
```

Logs presentation counts and latest pickup date. SELECT * reads the log back for verification.

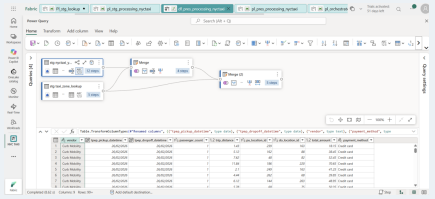
Power BI dashboard

Dashboard screenshot at full scale, with the Dataflow and workspace screenshots alongside

POWER BI REPORT

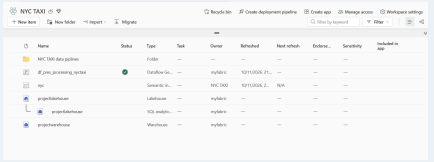


DATAFLOW GEN2



Power Query merges trip data with taxi-zone lookup values.

FABRIC WORKSPACE



Workspace assets include the Lakehouse, Warehouse, pipelines, and semantic model.